

Troubleshooting Lab Workbook

Hands-on practice in finding what is actually wrong — from reading a complaint carefully to tracing a cascade across servers. Written for people new to IT, worked in pairs.

How to use this workbook

Every lab is done by two people. One is the **learner**, who diagnoses. The other is the **partner**, who breaks something in secret and plays the user who reports it. Swap roles each lab.

LEARNER

Reads the lab page. Works the tasks in order. Fills in a Diagnostic Log (Appendix A). Answers the debrief questions.

PARTNER

Reads only the matching fault card (Appendix B). Injects the fault while the learner looks away. Answers questions as a user would — truthfully, but no hints.

GROUND RULES

1. Snapshot before every lab. Restore after. Never carry a fault into the next lab.
2. Change one thing at a time, and write it down before you do it.
3. Evidence before theory. If you can't point at a log line, a number, or a test result, it's a guess.
4. Commands in this workbook are examples. Use whatever tool shows the same evidence on your system.
5. Getting stuck is part of the exercise. Log where you stopped and what you would try next.

The lab environment

Any hypervisor works if it supports **snapshots** and an **isolated internal network** — VirtualBox, VMware, Hyper-V, or Proxmox. Plan for a host with 32 GB RAM and 300 GB free disk. You build it all in Lab 3; Labs 1–2 need only the notes app and paper.

HOST	ADDRESS	OS	ROLE
gw01	10.10.0.1	Debian 12	Router, DHCP + DNS (dnsmasq), NAT for updates
web01	10.10.0.10	Ubuntu 24.04	Notes app: nginx → gunicorn; later k3s
db01	10.10.0.20	Ubuntu 24.04	PostgreSQL 16
idp01	10.10.0.30	Ubuntu 24.04	Identity provider (Keycloak), SSO for the app
mon01	10.10.0.40	Ubuntu 24.04	Metrics + logs (Prometheus, Grafana, Loki)
win01	10.10.0.100	Windows 11 eval	User workstation, Sysmon
atk01	10.99.0.66	Debian 12	Lab 16 only, on isolated segment <code>redlab</code>

SAFETY

Keep labnet (10.10.0.0/24) host-only. Never run a fault on a machine you use for real work or on a shared network. Lab 16 runs only on `redlab`, with no route anywhere. Lab 7 involves real hardware — unplug before opening a case, and never open a power supply.

Labs

Difficulty: ■□□ foundation · ■■□ core · ■■■ stretch. Do them in order; each assumes the habits of the ones before.

NO.	LAB	TIME	LEVEL
01	Symptom, Cause, Fix	30 min	■□□
02	Walk the Full Diagnostic Loop	60 min	■□□
03	Build the Troubleshooting Lab	4 h	■■□
04	Mindset Under the Clock	45 min	■■□
05	Isolation: Halve the Search Space	60 min	■■□
06	Read What the System Tells You	60 min	■■□
07	Hardware: Power First, Then Substitute	90 min	■■□
08	Operating Systems: Boot, Slowdown, Crash	90 min	■■□
09	Networks: Bottom-Up	75 min	■■□
10	Applications and Services	75 min	■■□
11	Databases and the Silent Wrong Answer	90 min	■■■
12	Identity and Access	75 min	■■■
13	Cloud, Virtualization, Containers	90 min	■■■
14	Hunt the Intermittent Fault	3 h	■■■
15	Trace the Cascade to Its Root	75 min	■■■
16	Fault or Attack?	90 min	■■■
17	Capstone: The Blind Incident	2 h	■■■

Symptom, Cause, Fix

Separate what you observe from what is wrong from what you would do — before touching anything.

SETUP

- No systems needed. Paper or a text file.
- Case A: “The website has been down since this morning.”
- Case B: “My laptop got really slow after lunch — probably a virus.”
- Case C: “The payroll export shows wrong totals for three employees.”

FAULT

None. This is a paper exercise.

TASKS

- 01** For each case, rewrite the complaint as a pure symptom: only what was observed, by whom, and when. Strip out every interpretation.
- 02** Underline any words in the original complaint that are already a guess at the cause (“down”, “virus”).
- 03** List at least four possible causes per case, each in a different layer (hardware, OS, network, application, data, identity).
- 04** Beside each cause, write the fix it would imply. Note how different the fixes are.
- 05** Write the single question you would ask first that best separates the causes from each other.

HAND IN

A three-column table (Symptom · Possible causes · Implied fixes) for each case, plus your first discriminating question.

COMMON TRAP

Writing “the server crashed” as the symptom. That is a cause-guess; the symptom is “the page did not load”.

Which fix would you have applied if you had skipped this exercise? Which cause would it have missed?

Did any two causes share a fix? What does that tell you about fixes as evidence?

Which complaint contained the most hidden assumptions?

Walk the Full Diagnostic Loop

Diagnose a broken service by performing — and writing down — every stage of the loop: define, gather, hypothesize, test, act, verify.

SETUP

- web01 at snapshot baseline; the notes app reachable at `http://10.10.0.10/`.
- One blank Diagnostic Log (Appendix A).
- Partner out of sight while the fault is introduced.

FAULT

Your partner injects it from Fault Card 02 (Appendix B). Don't read the card.

TASKS

- 01** Define: write a precise problem statement — who is affected, exact error, since when, and whether it reproduces every time.
- 02** Gather: collect evidence before theorizing. Status code, nginx error log, service status, listening ports.
- 03** Hypothesize: write one specific, testable explanation in the form “If X is the cause, then I will observe Y.”
- 04** Test: run the check that would prove the hypothesis false. Do not change anything yet.
- 05** Act: make exactly one change that addresses the confirmed cause.
- 06** Verify: confirm the symptom is gone, the logs are clean, and nothing else broke. Restore snapshot.

EXAMPLE COMMANDS

ADAPT TO YOUR TOOLS

```
curl -sI http://10.10.0.10/
sudo tail -n 50 /var/log/nginx/error.log
ss -ltnp | grep -E ':800[01]'
systemctl status gunicorn-notes
```

HAND IN

A Diagnostic Log with all six stages filled in, each with a timestamp.

COMMON TRAP

Restarting nginx “to see if it helps” before a hypothesis exists.

DEBRIEF — WRITE YOUR ANSWERS

Which stage did you most want to skip?

Did your test have a result that could have disproved your hypothesis?

How did you verify “nothing else broke”?

Build the Troubleshooting Lab

Stand up the lab environment used by every later exercise, then diagnose a fault you did not see introduced.

SETUP

- Hypervisor host (see The Lab Environment).
- ISOs: Ubuntu Server 24.04, Debian 12, Windows 11 Enterprise evaluation.
- A lab notebook — paper or a Git repo.

FAULT

Your partner injects it from Fault Card 03 (Appendix B). Don't read the card.

TASKS

- 01** Create the host-only network labnet (10.10.0.0/24) with no DHCP from the hypervisor.
- 02** Build gw01 first (routing, dnsmasq for DHCP and DNS, NAT for updates), then web01, db01, idp01, mon01, win01.
- 03** Deploy the notes app on web01 (nginx → gunicorn → PostgreSQL on db01). Confirm it works from win01.
- 04** Install evidence tooling: node_exporter and promtail on Linux hosts, Sysmon on win01, Prometheus + Grafana + Loki on mon01.
- 05** Take a snapshot named baseline on every VM. Record names in your inventory sheet.
- 06** Have your partner (or scripts/breakit.sh) inject one fault while you look away. Diagnose it with the Lab 2 loop.
- 07** Restore baseline on the affected VM, verify health, then repeat with a second fault.

EXAMPLE COMMANDS

ADAPT TO YOUR TOOLS

```
VBoxManage snapshot web01 take baseline --description "clean build"  
VBoxManage snapshot web01 restore baseline  
# Proxmox: qm snapshot 110 baseline && qm rollback 110 baseline
```

HAND IN

Lab inventory sheet (hostname, IP, role, credentials location, snapshot names) and two completed Diagnostic Logs.

COMMON TRAP

Skipping baseline snapshots. Without a known-good state, every later lab starts contaminated.

DEBRIEF — WRITE YOUR ANSWERS

How long did a full break → diagnose → restore cycle take?

Which evidence source was most useful? Which did you not know how to read yet?

What will you add to the inventory sheet next time?

Mindset Under the Clock

Practise calm, one-change-at-a-time, what-changed and assume-nothing habits under time pressure, and catch yourself wanting to skip them.

SETUP

- web01 and db01 at baseline.
- A visible 20-minute timer.
- Two blank sheets: Change Log and Shortcut Log.

FAULT

Your partner injects it from Fault Card 04 (Appendix B). Don't read the card.

TASKS

- 01** Start the timer. Before touching anything, ask your partner: "What changed, and when?" Write down the answer verbatim.
- 02** List every assumption you are making (e.g. "the DB is up", "the app config is unchanged"). Check each one and mark it verified or false.
- 03** Make one change at a time. Log each with a timestamp, the reason, and the result.
- 04** Every time you feel the urge to change two things or skip a test, write it in the Shortcut Log instead of doing it.
- 05** Stop at 20 minutes whether solved or not. Restore snapshots.

HAND IN

Change Log and Shortcut Log with timestamps.

COMMON TRAP

Chasing the loudest error in the log instead of the one that matches the timeline.

How many entries are in your Shortcut Log? Which one would have hurt most?

Did the red-herring error pull your attention? When did you rule it out, and with what evidence?

What was your breathing and pace like at minute 15 versus minute 2?

Isolation: Halve the Search Space

Narrow a problem with many possible causes to its root, eliminating roughly half the possibilities with each test.

SETUP

- All VMs at baseline.
- Draw the request path: win01 → gw01 → web01 (nginx → gunicorn) → db01 (PostgreSQL).

FAULT

Your partner injects it from Fault Card 05 (Appendix B). Don't read the card.

TASKS

- 01 Scope first: does it fail from win01 only, or from web01 itself too? Write what that eliminates.
- 02 Pick the midpoint of the path. Test directly at it (e.g. curl localhost:8000 on web01). Cross out the half that is proven healthy.
- 03 Repeat at the midpoint of the remaining half until one component is left.
- 04 Use works-here-not-there: compare the failing request with a similar one that works (another host, another port, another user).
- 05 Confirm with substitution: swap in a known-good component (e.g. a throwaway postgres container on web01) — change only the one variable.
- 06 Fix, verify from win01, restore snapshots.

EXAMPLE COMMANDS

ADAPT TO YOUR TOOLS

```
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:8000/ # on web01
nc -zv 10.10.0.20 5432
psql -h 10.10.0.20 -U notes -c 'select 1'
docker run --rm -e POSTGRES_PASSWORD=x -p 5433:5432 postgres:16
```

HAND IN

A path diagram showing each test, its result, and the half it eliminated.

COMMON TRAP

Checking components in the order you happen to think of them rather than splitting the path.

DEBRIEF — WRITE YOUR ANSWERS

How many tests did it take? How many would a random check of each component have taken?

Did your substitution introduce any new variable (version, credentials, network)?

At which step did scope alone do the most work?

Read What the System Tells You

Diagnose a fault using only the evidence the system provides — errors, logs, metrics — before writing any hypothesis.

SETUP

- All VMs at baseline; Grafana dashboards on mon01 open.
- Rule: no hypothesis may be written until step 5.

FAULT

Your partner injects it from Fault Card 06 (Appendix B). Don't read the card.

TASKS

- 01 Reproduce: find the exact action that fails and one that works. Write both.
- 02 Capture the exact error text from the browser and from the app log. Copy, don't paraphrase.
- 03 Build a timeline: pull logs from web01 and db01 for the same window. Line them up by timestamp.
- 04 Read the metrics for the window: disk, CPU, memory, connections. Note anything that changed shape.
- 05 Now write a hypothesis. Each claim must cite a specific log line or metric.
- 06 Test, fix, verify. Then confirm the evidence pointed at the true cause all along.

EXAMPLE COMMANDS

ADAPT TO YOUR TOOLS

```
journalctl -u gunicorn-notes --since "30 min ago" --no-pager
sudo tail -n 100 /var/log/postgresql/postgresql-16-main.log
df -h; df -i
# Windows: Get-WinEvent -LogName System -MaxEvents 50
```

HAND IN

A timeline table (time · host · source · entry) with the causal entry highlighted.

COMMON TRAP

Reading only the last line of a log. The cause is usually earlier than the loudest symptom.

DEBRIEF — WRITE YOUR ANSWERS

Which log line contained the answer? How many lines of noise surrounded it?

Did the working-versus-failing reproduction narrow the cause before you read any logs?

What evidence did you not know existed before this lab?

Hardware: Power First, Then Substitute

Diagnose staged physical faults from no-power to a failing drive, applying power-first and substitution.

SETUP

- Bench PC with known-good spares: PSU, one RAM stick, SATA cable, drive.
- PSU tester or multimeter, ESD strap, a USB stick with memtest86+ and a live Linux image.
- Safety: unplug before touching internals. Never open a power supply.

FAULT

Your partner injects it from Fault Card 07 (Appendix B). Don't read the card.

TASKS

- 01** Power first: wall outlet, power strip, cable, PSU switch, PSU tester. Record each check.
- 02** No-boot: watch for fans, LEDs, POST beeps or codes. Write down which boot stage was reached.
- 03** Strip to minimum config (board, CPU, one RAM stick, PSU). Add components back one at a time.
- 04** Substitute one suspect at a time with its known-good spare. Never swap two parts at once.
- 05** For the drive: read SMART data and temperatures. Decide whether it is failing or has failed, and why that matters.
- 06** Restore the bench and label any bad parts.

EXAMPLE COMMANDS

ADAPT TO YOUR TOOLS

```
sudo smartctl -a /dev/sda
sudo smartctl -t short /dev/sda
sudo sensors
sudo dmesg -T | grep -iE 'ata|i/o error|reset'
```

HAND IN

A per-fault record: first observation, checks in order, the substitution that localized it.

COMMON TRAP

Swapping RAM and PSU together — it works, and you have learned nothing.

DEBRIEF — WRITE YOUR ANSWERS

How many checks did power-first save on fault 1?

What evidence separated “failing” from “failed” on the drive?

Where might an intermittent cable fault have been blamed on the OS?

Operating Systems: Boot, Slowdown, Crash

Diagnose staged OS faults and prove each one lives in the OS layer, not the hardware below or the application above.

SETUP

- web01 at baseline, console access via the hypervisor.
- A second terminal for measurement.

FAULT

Your partner injects it from Fault Card 08 (Appendix B). Don't read the card.

TASKS

- 01** Boot: identify the stage that failed — firmware, bootloader, kernel, init, services. Write what that stage rules in and out.
- 02** Read the previous boot's journal and the failed-units list.
- 03** Slowdown: turn “slow” into numbers — CPU, memory, swap, I/O wait, top processes. Identify the process by behaviour, not just by name.
- 04** Crash: read kernel messages. Separate an OOM kill (software) from machine-check or I/O errors (hardware).
- 05** For each fault, find what changed: package history, unit overrides, fstab edits.
- 06** Fix one at a time, verify, restore snapshot.

EXAMPLE COMMANDS

ADAPT TO YOUR TOOLS

```
journalctl -b -1 -p err --no-pager
systemctl --failed
vmstat 1 5; iostat -x 1 3; pidstat 1 5
sudo dmesg -T | grep -iE 'oom|mce|hardware error'
systemctl cat gunicorn-notes
less /var/log/apt/history.log
```

HAND IN

For each fault: the layer it lives in and the specific evidence that rules out the layers above and below.

COMMON TRAP

Blaming hardware for slowness that a single process explains.

DEBRIEF — WRITE YOUR ANSWERS

What made the disguised process visible?

How would you tell an OOM kill from bad RAM if both looked like random crashes?

Which fault would the recent-change question have found fastest?

Networks: Bottom-Up

Diagnose staged network faults with the bottom-up sequence — link, address, gateway, DNS, destination — and the right tool at each step.

SETUP

- All VMs at baseline; work from win01 and web01.

FAULT

Your partner injects it from Fault Card 09 (Appendix B). Don't read the card.

TASKS

- 01** Ask first: is it me, the path, or the far end? Test the same destination from a second host.
- 02** Link: is the interface up with carrier?
- 03** Address: correct IP, mask, gateway?
- 04** Gateway: can you reach it by IP? Beyond it by IP?
- 05** Name: does the destination resolve? If it works by IP but not by name, you are looking at DNS.
- 06** Port: is the service port open at the destination?
- 07** Slowdown: separate bandwidth, latency, and loss. Localize to a hop.

EXAMPLE COMMANDS

ADAPT TO YOUR TOOLS

```
ip -br a; ip route; ping -c4 10.10.0.1
dig notes.lab @10.10.0.1
nc -zv 10.10.0.10 80
mtr -rwc 50 10.10.0.10
iperf3 -c 10.10.0.10
# Windows: ipconfig /all; Test-NetConnection 10.10.0.10 -Port 80; Resolve-
DnsName notes.lab
```

HAND IN

A layer ladder for each fault showing the first layer that failed and the tool that proved it.

COMMON TRAP

Declaring “no connectivity” when only name resolution is broken.

DEBRIEF — WRITE YOUR ANSWERS

Which fault looked like “no internet” but was not?

What did mtr show that ping alone could not?

What does a successful ping prove, and what does it not prove?

Applications and Services

Diagnose an app that won't start, a broken dependency, and a performance problem — separating the app from the layers around it.

SETUP

- web01 and db01 at baseline.
- A load generator (hey or k6) on win01 or mon01.

FAULT

Your partner injects it from Fault Card 10 (Appendix B). Don't read the card.

TASKS

- 01 One app or everything? Check a second service on the same host.
- 02 Read the app's own errors: unit status, journal, app log.
- 03 Write the dependency chain (config → DNS → DB → disk...). Test each link directly, not through the app.
- 04 Performance: time each stage of a request — connect, first byte, total — and compare to app-reported request time.
- 05 Compare a working environment with the broken one: diff config and environment variables.
- 06 Fix, verify under load, restore.

EXAMPLE COMMANDS

ADAPT TO YOUR TOOLS

```
systemctl status gunicorn-notes; journalctl -u gunicorn-notes -n 50
curl -o /dev/null -s -w 'connect %{time_connect} ttfb %{time_starttransfer}
total %{time_total}\n' http://10.10.0.10/notes
diff <(sort /etc/notes/app.env) <(sort ~/known-good/app.env)
```

HAND IN

For each fault: the dependency chain with the broken link marked, and the evidence.

COMMON TRAP

Blaming the network for a 500 error. A 500 means the network worked.

DEBRIEF — WRITE YOUR ANSWERS

Which fault would a user have reported as “the network is down”?

Where was the time going in the slow case — network, app queue, or DB?

What did the diff reveal that reading the file did not?

Databases and the Silent Wrong Answer

Diagnose connection, performance, concurrency, and — the dangerous one — a query that runs cleanly and returns wrong data.

SETUP

- db01 at baseline, seeded with the reports schema (orders, order_lines, daily_totals view).
- Two psql sessions open.

FAULT

Your partner injects it from Fault Card 11 (Appendix B). Don't read the card.

TASKS

- 01** Connection: test reachability, then authentication, then authorization. Read the server log for the refusal reason.
- 02** Performance: get the actual plan with timing. Find the step where estimated and actual rows diverge or a sequential scan dominates.
- 03** Wrong results: compute the expected answer independently from raw rows for a small sample. Compare to the report.
- 04** Bisect the query: run its pieces separately until the discrepancy appears.
- 05** Concurrency: reproduce the deadlock; read locks and waits while it happens.
- 06** For each fault, check schema and config history for the change.

EXAMPLE COMMANDS

ADAPT TO YOUR TOOLS

```
pg_isready -h 10.10.0.20
EXPLAIN (ANALYZE, BUFFERS) SELECT ...;
SELECT pid, state, wait_event_type, query FROM pg_stat_activity;
SELECT * FROM pg_locks WHERE NOT granted;
SHOW timezone;
```

HAND IN

A written trace for the wrong-results fault: expected value, actual value, the query fragment where they diverge, and the cause.

COMMON TRAP

Diagnosing a slow query without reading its plan.

DEBRIEF — WRITE YOUR ANSWERS

How would anyone have noticed the wrong totals in production?

What did the plan show that the query text did not?

Which of these four faults is a design flaw, not a bug?

Identity and Access

Diagnose staged identity faults and name which of the four failed: authentication, authorization, provisioning, or federation.

SETUP	• idp01 running Keycloak, realm lab; notes app uses OIDC login. • Users alice (editor) and bob (editor via group). Admin console access.
FAULT	Your partner injects it from Fault Card 12 (Appendix B). Don't read the card.

TASKS

- 01 Classify first: did the identity fail to prove itself, get denied, not exist correctly, or fail at a trust boundary?
- 02 Read the IdP event log for the user and time window.
- 03 Decode the token the app received. Check issuer, audience, expiry, and role/group claims.
- 04 For denials, trace effective access: every path (direct, group, nested group, role) from user to permission.
- 05 For federation, check clocks, secrets, and signing keys on both sides.
- 06 Verify fixes as the user, not as the admin.

EXAMPLE COMMANDS

ADAPT TO YOUR TOOLS

```
date -u; timedatectl    # on idp01 and web01
echo "$TOKEN" | cut -d. -f2 | base64 -d 2>/dev/null | jq .
# Keycloak: Realm → Events → Login events; Users → bob → Role mapping →
effective
```

HAND IN

For each fault: which of the four A's failed and the evidence that proved it.

COMMON TRAP

Loosening resource permissions when the real problem is a missing claim.

DEBRIEF — WRITE YOUR ANSWERS

Which fault was invisible at the resource?

Why did alice's session survive her password reset, and why does that matter in an incident?

Which usual suspect caused fault 1?

Cloud, Virtualization, Containers

Diagnose faults where the infrastructure is abstracted and determine whose layer — guest, host, platform, or yours — the problem lives in.

SETUP

- k3s on web01 with the notes app deployed as a pod.
- Optional: AWS free-tier account with a \$5 budget alarm set before starting.

FAULT

Your partner injects it from Fault Card 13 (Appendix B). Don't read the card.

TASKS

- 01 Whose problem is it? Check the provider or hypervisor status before your own config.
- 02 Contention: in the guest, look at steal time. On the host, look at CPU ready and overcommit ratio.
- 03 Crash loop: capture evidence before it vanishes — previous container logs and events — before deleting anything.
- 04 Scheduling: read pod events for the scheduler's reason.
- 05 Cloud access: use the platform's own audit trail and policy simulator to find the exact denied action.
- 06 Fix, verify, restore.

EXAMPLE COMMANDS

ADAPT TO YOUR TOOLS

```
top    # watch %st
kubectl get pods -A -o wide
kubectl describe pod <pod>
kubectl logs <pod> --previous
kubectl get events --sort-by=.lastTimestamp
# AWS: CloudTrail event history; IAM policy simulator
```

HAND IN

A layer diagram for each fault marking guest / host / platform / your config, with the responsible layer circled.

COMMON TRAP

Deleting a crash-looping pod to “reset it”
— the evidence goes with it.

DEBRIEF — WRITE YOUR ANSWERS

What did steal time tell you that CPU utilization did not?

What evidence would you have lost by deleting the crashing pod?

How would a sudden cost spike have looked in this lab?

Hunt the Intermittent Fault

Find the pattern in a fault you cannot force, and set a trap that captures the system's state at the moment it happens.

SETUP

- web01 and db01 at baseline. Plan for the lab to run over one to two days.
- A shared occurrence sheet your partner or a synthetic check can write to.

FAULT

Your partner injects it from Fault Card 14 (Appendix B). Don't read the card.

TASKS

- 01 Record every occurrence: time, duration, user action, what else was running.
- 02 Add a synthetic check: every 10 seconds, write a note and log latency and result to a file.
- 03 After 5+ occurrences, look for what they share — time, load, sequence, host, job.
- 04 Set a trap: on the first failed check, capture connections, processes, firewall rules, and the last minute of logs.
- 05 Analyze the capture. Form and test a hypothesis.
- 06 Fix. Declare it fixed only after a clean period at least three times the longest gap you observed.

```
while true; do
  if ! curl -sf -m 3 -X POST http://10.10.0.10/notes -d 'x=1' >/dev/null;
  then
    ts=$(date +%s)
    ssh db01 "sudo iptables -S; ss -tnp; ps aux --sort=-%cpu | head" >
    trap-$ts.txt
  fi
  sleep 10
done
```

HAND IN

Occurrence table, the trap script, one capture file, and the reasoning that linked them.

COMMON TRAP

Declaring it fixed because it has been quiet for an hour.

DEBRIEF — WRITE YOUR ANSWERS

How many occurrences did it take before a pattern appeared?

What did the capture show that no after-the-fact log could have?

What evidence would justify saying “fixed”?

Trace the Cascade to Its Root

Diagnose a chain of failures by building the timeline and tracing backward, instead of fixing the loudest symptom.

SETUP	• All VMs at baseline with synchronized clocks (verify first). • Grafana and Loki on mon01.
FAULT	Your partner injects it from Fault Card 15 (Appendix B). Don't read the card.

TASKS

- 01 List every symptom with the time it was first seen.
- 02 Build one timeline across all hosts from logs and alerts, sorted by timestamp.
- 03 Draw the dependency graph between the failing components.
- 04 Find the earliest failure that has no failing upstream dependency. That is the root candidate.
- 05 Fix only the root. Watch downstream components recover without touching them.
- 06 Anything that does not recover is a separate fault. Diagnose it on its own.

EXAMPLE COMMANDS

ADAPT TO YOUR TOOLS

```
for h in web01 db01 idp01 mon01; do
  ssh $h "journalctl --since '-20 min' -o short-iso -p warning --no-pager" |
  sed "s/^/$h /"
done | sort -k2
```

HAND IN

The merged timeline with the root event marked, and a dependency graph showing the cascade.

COMMON TRAP

Restarting the loudest service. It comes back, fails again, and the root stays hidden.

DEBRIEF — WRITE YOUR ANSWERS

Which symptom was loudest? How far downstream was it?

Did everything recover after the root fix? What didn't, and why?

What would restarting the app first have achieved?

Fault or Attack?

Diagnose an ambiguous problem, decide whether it is malfunction or malice, and pivot to evidence preservation when it proves to be an attack.

SETUP

- Isolated segment redlab only: a disposable clone of web01 and atk01. Confirm redlab has no route to labnet or the internet.
- External evidence share (a separate virtual disk) and a chain-of-custody sheet.

FAULT

Your partner injects it from Fault Card 16 (Appendix B). Don't read the card.

TASKS

- 01 Start as a normal performance diagnosis. Measure and identify the process.
- 02 List every anomaly that does not fit a malfunction: unknown accounts, odd timing, gaps in logs, unexpected cron entries.
- 03 Decide: malfunction or malice? Write down the specific signals behind your call.
- 04 If malice: stop troubleshooting reflexes. Do not reboot, kill, delete, or patch.
- 05 Preserve: take a live snapshot with memory, capture volatile state to the external share, and hash every file.
- 06 Escalate: write a handoff for the incident-response process with what you saw, what you touched, and when.

EXAMPLE COMMANDS

ADAPT TO YOUR TOOLS

```
getent passwd | awk -F: '$3>=1000'  
sudo ls -la /etc/cron.d /var/spool/cron/crontabs  
last -F; sudo lastb | head  
ss -tnp > /mnt/evidence/ss.txt  
sha256sum /mnt/evidence/* > /mnt/evidence/SHA256SUMS  
# Hypervisor: live snapshot including memory
```

HAND IN

Anomaly list, the malfunction-vs-malice call with reasons, the evidence folder with hashes, and a one-page handoff.

COMMON TRAP

Killing the process and deleting the cron job. The symptom stops and the evidence is gone.

DEBRIEF — WRITE YOUR ANSWERS

Which single signal tipped you from “fault” to “attack”?

What would a reboot have destroyed?

List everything you touched on the host. Could any of it be mistaken for attacker activity?

Capstone: The Blind Incident

Run a full incident end to end — scope, diagnose, fix, verify, and write it up — against faults you have not seen, with no hints.

SETUP

- All VMs at baseline.
- Partner acts as the reporting user and the only source of “what changed”.
- 60-minute time box for diagnosis; 30 minutes for the write-up.

FAULT

Your partner injects it from Fault Card 17 (Appendix B). Don’t read the card.

TASKS

- 01 Open a Diagnostic Log. Write the problem statement from the report alone.
- 02 Scope: who, what, since when, every time or sometimes.
- 03 Work the loop. Use isolation to split the path and evidence to rank hypotheses.
- 04 When one fix lands, re-verify the original symptom. Decide whether anything remains.
- 05 Restore everything to baseline and confirm health from win01.
- 06 Write a blameless post-incident review: timeline, root cause(s), what detected it, what slowed you down, and one preventive action per cause.

HAND IN

Diagnostic Log plus a one-page post-incident review.

COMMON TRAP

Stopping after the first fix because the symptom got better, not gone.

DEBRIEF — WRITE YOUR ANSWERS

Did you find both faults? How did you know there was a second one?

Which earlier lab's habit helped most today?

What would you add to monitoring so the next person finds this in five minutes?

Diagnostic Log

Copy this page for every lab. Timestamp each stage as you enter it.

LAB	DATE	LEARNER
PARTNER	START	END

01 Define Who is affected, the exact error, since when, every time or sometimes.	time ____:____
02 Gather Evidence before theory: errors, logs, status, metrics.	time ____:____
03 Hypothesize If X is the cause, then I will observe Y.	time ____:____
04 Test The check that could prove it false. Change nothing yet.	time ____:____

05 Act One change. What, why, and how to undo it.	time ____:____
06 Verify Symptom gone, logs clean, nothing else broken.	time ____:____

Fault cards

Print these pages separately, or tear them out before handing over the workbook. Read the card, snapshot, inject the fault, then call the learner in. When the lab ends, restore the baseline snapshot and confirm the app works from win01.

CARD 02

WALK THE FULL DIAGNOSTIC LOOP

On web01, edit `/etc/nginx/sites-enabled/notes` and change `proxy_pass` port 8000 to 8001. Run `sudo nginx -s reload`. Symptom: 502 Bad Gateway.

Reset: restore every touched VM to baseline. Don't explain the fault until the debrief.

CARD 03

BUILD THE TROUBLESHOOTING LAB

After baselines exist, pick two, one at a time: (1) `sudo systemctl stop postgresql` on db01; (2) set a wrong nameserver in web01's netplan and apply; (3) on web01, `sudo fallocate -l [free space] /var/fill.img` to fill `/var`.

Reset: restore every touched VM to baseline. Don't explain the fault until the debrief.

CARD 04

MINDSET UNDER THE CLOCK

(1) Change the DB password in `/etc/notes/app.env` on web01 and restart the app. (2) Red herring: add a cron job that runs `logger -p user.err "ERROR disk check failed"` every minute. If asked what changed, say only: "There was a deploy at [time]."

Reset: restore every touched VM to baseline. Don't explain the fault until the debrief.

CARD 05

ISOLATION: HALVE THE SEARCH SPACE

Choose one, secretly: (a) on db01, `sudo iptables -I INPUT -p tcp --dport 5432 -s 10.10.0.10 -j REJECT`; (b) change `DATABASE_HOST` in `app.env` to a wrong IP; (c) remove the default route on gw01. Symptom for all: the notes page fails from win01.

Reset: restore every touched VM to baseline. Don't explain the fault until the debrief.

CARD 06**READ WHAT THE SYSTEM TELLS YOU**

On db01, fill the PostgreSQL data volume: `sudo fallocate -l [free space minus 50M] /var/lib/postgresql/fill.img`. Reads keep working; new notes fail with a generic 500.

Reset: restore every touched VM to baseline. Don't explain the fault until the debrief.

CARD 07**HARDWARE: POWER FIRST, THEN SUBSTITUTE**

Stage in sequence, resetting between each: (1) switch off the PSU rocker or half-seat the 24-pin connector; (2) replace one RAM stick with a known-bad one, or seat it partially; (3) fit a drive with SMART errors, or a SATA cable that is loose enough to drop intermittently.

Reset: restore every touched VM to baseline. Don't explain the fault until the debrief.

CARD 08**OPERATING SYSTEMS: BOOT, SLOWDOWN, CRASH**

Stage one at a time: (1) in `/etc/fstab` add a line mounting a nonexistent UUID without `nofail`, then reboot; (2) run `stress-ng --vm 2 --vm-bytes 90% renamed as kworkerd via exec -a`; (3) add `MemoryMax=60M` to the `unicorn-notes` unit and restart so it is OOM-killed under light load.

Reset: restore every touched VM to baseline. Don't explain the fault until the debrief.

CARD 09**NETWORKS: BOTTOM-UP**

One at a time: (1) give win01 a static IP with a wrong default gateway; (2) `sudo systemctl stop dnsmasq` on gw01; (3) on gw01, `sudo tc qdisc add dev eth1 root netem delay 200ms loss 5%`.

Reset: restore every touched VM to baseline. Don't explain the fault until the debrief.

CARD 10**APPLICATIONS AND SERVICES**

One at a time: (1) introduce a typo in a variable name in `/etc/notes/app.env` and restart; (2) stop `postgresql` on db01; (3) set `unicorn workers=1` and run `hey -z 120s -c 30` against `/notes`.

Reset: restore every touched VM to baseline. Don't explain the fault until the debrief.

CARD 11**DATABASES AND THE SILENT WRONG ANSWER**

Stage one at a time: (1) delete the web01 line from pg_hba.conf and reload; (2) drop the index on order_lines(order_id) and load 1M rows; (3) ALTER DATABASE notes SET timezone='Pacific/Auckland' so daily_totals shifts orders across dates; (4) give the learner the deadlock script (two sessions updating rows in opposite order).

Reset: restore every touched VM to baseline. Don't explain the fault until the debrief.

CARD 12**IDENTITY AND ACCESS**

One at a time: (1) disable NTP on idp01 and set the clock 6 minutes ahead (TOTP and token validity fail); (2) remove the editors role from bob's group mapping; (3) regenerate the notes client secret in Keycloak without updating the app; (4) reset alice's password but leave her existing session active.

Reset: restore every touched VM to baseline. Don't explain the fault until the debrief.

CARD 13**CLOUD, VIRTUALIZATION, CONTAINERS**

One at a time: (1) on the hypervisor, pin a stress-ng --cpu VM to the same cores as web01 (noisy neighbour); (2) deploy notes with a liveness probe on the wrong path so it crash-loops; (3) set a pod resource request of 64Gi so it will not schedule; (4, optional cloud) remove s3:GetObject from the app role's policy.

Reset: restore every touched VM to baseline. Don't explain the fault until the debrief.

CARD 14**HUNT THE INTERMITTENT FAULT**

Install a systemd timer on db01 that runs at a randomized time within each hour but only when a report job is running: it inserts iptables -I INPUT -p tcp --dport 5432 -j DROP for 45 seconds, then removes the rule. Tell the learner only: "Notes sometimes fails to save."

Reset: restore every touched VM to baseline. Don't explain the fault until the debrief.

CARD 15

TRACE THE CASCADE TO ITS ROOT

Stop dnsmasq on gw01. Within minutes the app can't resolve db01.lab, SS0 can't reach idp01.lab, health checks fail, and alerts fire everywhere. Optional second, unrelated fault: an expired TLS certificate on mon01.

Reset: restore every touched VM to baseline. Don't explain the fault until the debrief.

CARD 16

FAULT OR ATTACK?

From atk01: log in to the clone with a weak password, create user svc-backup with sudo, add a cron job every 5 minutes running a CPU burner (yes > /dev/null for 90 s), and delete a range of lines from /var/log/auth.log. Report to the learner: "web01 is slow at odd intervals."

Reset: restore every touched VM to baseline. Don't explain the fault until the debrief.

CARD 17

CAPSTONE: THE BLIND INCIDENT

Combine two faults from different layers that interact, e.g. Lab 12 fault 1 (skewed clock on idp01) plus Lab 10 fault 3 (one gunicorn worker) under load. Report only: "Login is slow and sometimes fails, since about 9:00."

Reset: restore every touched VM to baseline. Don't explain the fault until the debrief.